

AfricaHackon Cybersecurity Lab

Network Forensics & Firewall Hardening

Assignment 1: Keylogger Incident Investigation • Assignment 2: UFW Firewall Configuration

Prepared by: Nelson Ngumo

Date: 24 August 2026

Environment: Wireshark (Noobs Keylogger.pcap) & UFW on Linux VM

Assignment 1 — Network Forensics: Keylogger Investigation

1. Objective

Investigate a suspected keylogger infection within a monitored network using Wireshark. The goal is to identify the infected host, trace its communication to the attacker's server, determine the frequency and content of the exfiltrated data, identify the attacker, and recover any files transmitted.

2. Tools & Environment

- Wireshark (GUI analysis of Noobs Keylogger.pcap)
- Capture source: github.com/nipunjaswal/networkforensics — Ch1/Noobs Keylogger/Noobs Keylogger.pcap
- Assumed scenario: analyst connected to a network mirror port; PCAP used here to simulate that captured traffic
- Live-capture equivalent (mirror/SPAN port, no ready-made PCAP available): from the analyst host connected to the mirror port, run `sudo dumptcap -i <iface> -w capture.pcapng` (or `sudo tcpdump -i <iface> -w capture.pcap`) to passively sniff all traffic, then open the resulting file in Wireshark/tshark and apply the same filters and steps used below on Noobs Keylogger.pcap.

3. Methodology

- Statistics > Endpoints and Statistics > Conversations to identify the busiest internal host
- Filter traffic to/from the suspect host and inspect protocols in use
- Statistics > IO Graph to determine transmission interval / beaconing pattern
- Follow TCP/SMTP/HTTP Stream to inspect payload content and headers
- File > Export Objects to recover any transferred files

4. Findings Summary

Item	Value / Status
Capture file	Noobs Keylogger.pcap
Infected host (internal IP)	192.168.76.131 — appears in nearly every conversation (2,163 packets / 945 kB total across 25+ external peers)
Gateway / broadcast (noise, excluded)	192.168.76.1 (gateway), 192.168.76.2, 192.168.76.255 (broadcast), 224.0.0.251 (mDNS), 239.255.255.250 (SSDP)
C2 / exfil server	140.82.59.185:21 (FTP, vsFTPD 3.0.3) — confirmed via Follow TCP Stream on the port-21 control channel
Beacon frequency	~63 seconds between FTP delivery cycles (uploads observed at t=0.07s and t=63.40s)
Additional exfiltrated data	Active window/application titles, timestamps, logged-in Windows username ("nipun"), and visited URLs — not just raw keystrokes
Attacker indicators	FTP creds test_user / Nipun@123, drop directory "Test", server banner vsFTPD 3.0.3, tool signature string "Ardamax_FTP_Delivery" (Ardamax Keylogger)

Recovered files	Keys_2018-11-28_16-04-42.html (579 B) and Web_2018-11-29_11-28-13.html (621 B) — recovered via ftp-data export, see extracted_files/
-----------------	--

5. Infected System Identification

The Ethernet-level Endpoints view was checked first, but MAC addresses alone do not identify hosts across a routed network, so analysis moved to the IPv4 layer.

Endpoint Settings	Ethernet · 8	IPv4 · 33	IPv6 · 3	TCP · 91	UDP · 61				
Name resolution	Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes		
Limit to display filter	00:0c:29:ae:bfe3	2,192	960 kB	1,003	280 kB	1,189	680 kB		
	00:50:56:f6:76:7c	2,135	932 kB	1,189	680 kB	946	251 kB		
	01:00:5e:7f:ff:fa	28	14 kB	0	0 bytes	28	14 kB		
	33:33:00:00:00:0c	26	15 kB	0	0 bytes	26	15 kB		
	00:50:56:c0:00:08	5	1 kB	5	1 kB	0	0 bytes		
	33:33:00:01:00:02	3	471 bytes	0	0 bytes	3	471 bytes		
	ff:ff:ff:ff:ff:ff	3	525 bytes	0	0 bytes	3	525 bytes		
	01:00:5e:00:00:fb	2	649 bytes	0	0 bytes	2	649 bytes		

Figure 1 — Initial Ethernet Endpoints view (MAC-level; multicast/broadcast addresses present but not diagnostic on their own).

Endpoint Settings	Ethernet · 8	IPv4 · 33	IPv6 · 3	TCP · 91	UDP · 61															
Name resolution	Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Country	City	Latitude	Longitude	AS Number	AS Organization							
Limit to display filter	8.253.181.235	28	17 kB	18	16 kB	10	2 kB													
	8.253.224.254	20	17 kB	14	16 kB	0	734 bytes													
	13.73.162.220	23	6 kB	13	5 kB	10	1 kB													
	13.107.3.128	36	19 kB	22	18 kB	14	2 kB													
	13.107.21.200	596	270 kB	319	160 kB	277	118 kB													
	23.98.125.55	16	7 kB	8	6 kB	0	850 bytes													
	52.109.76.32	26	19 kB	17	17 kB	9	2 kB													
	52.109.120.21	31	17 kB	17	8 kB	14	8 kB													
	52.114.7.37	21	7 kB	12	5 kB	9	3 kB													
	52.114.128.43	63	23 kB	36	15 kB	27	8 kB													
	52.175.39.99	25	9 kB	13	7 kB	12	2 kB													
	52.229.207.60	105	67 kB	94	49 kB	91	17 kB													
	52.230.3.194	29	8 kB	17	5 kB	12	3 kB													
	52.230.85.180	50	15 kB	28	10 kB	22	5 kB													
	52.232.69.150	39	20 kB	26	20 kB	13	2 kB													
	65.55.163.76	249	178 kB	163	132 kB	86	46 kB													
	111.221.29.174	30	15 kB	17	11 kB	13	4 kB													
	117.18.232.200	39	24 kB	24	22 kB	15	2 kB													
	117.18.237.29	14	3 kB	6	2 kB	8	1 kB													
	140.82.59.185	88	7 kB	42	3 kB	46	4 kB													
	157.55.135.30	18	8 kB	9	7 kB	9	915 bytes													
	172.217.31.5	59	16 kB	27	13 kB	32	3 kB													
	172.217.31.14	12	2 kB	4	941 bytes	8	681 bytes													
	185.48.84.186	0	734 bytes	3	503 bytes	3	231 bytes													
	192.168.76.1	5	1 kB	5	1 kB	0	0 bytes													
	192.168.76.2	279	34 kB	141	23 kB	138	11 kB													
	192.168.76.131	2,163	945 kB	974	263 kB	1,189	680 kB													
	192.168.76.255	3	525 bytes	0	0 bytes	3	525 bytes													
	203.92.39.72	20	6 kB	10	5 kB	10	1 kB													
	203.92.39.73	20	6 kB	10	5 kB	10	1 kB													
	204.79.197.203	113	96 kB	79	93 kB	34	4 kB													
	224.0.0.251	2	649 bytes	0	0 bytes	2	649 bytes													
	239.255.255.250	28	14 kB	0	0 bytes	28	14 kB													

Figure 2 — IPv4 Endpoints view. 192.168.76.131 stands out with 2,163 packets and 945 kB across the largest number of distinct peers.

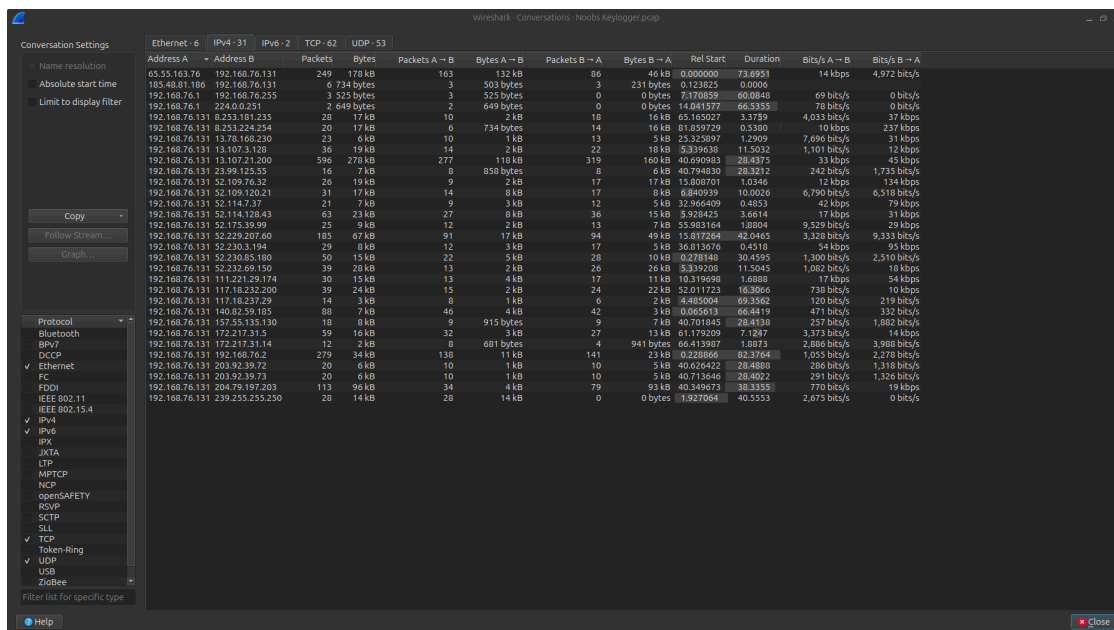


Figure 3 — IPv4 Conversations view. 192.168.76.131 appears as Address A in nearly every row, confirming it as the single internal host generating the bulk of outbound traffic.

Conclusion: 192.168.76.131 is identified as the infected system. Filter applied to isolate its traffic for further analysis:

```
ip.addr == 192.168.76.131
```

Addresses excluded as normal network noise: 192.168.76.1 (gateway), 192.168.76.2, 192.168.76.255 (broadcast), 224.0.0.251 (mDNS), 239.255.255.250 (SSDP). Several external IPs (13.107.x.x, 52.x.x.x Azure ranges, 204.79.197.203, 111.221.29.174, 157.55.135.130) correspond to legitimate Microsoft/Office telemetry and were provisionally deprioritized as C2 candidates.

6. Tracing Data to the C2 Server

CONFIRMED: Filtering `ip.src == 192.168.76.131 && !(tcp.port == 80 || tcp.port == 443)` isolates two short-lived TCP sessions to 140.82.59.185 on port 21 (FTP control) — conversations 51648↔140.82.59.185:21 and 51700↔140.82.59.185:21 in Statistics > Conversations — TCP tab, each immediately followed by a PASV data-channel session (ports 18439 and 5142) opened to the same host. Following the FTP control stream (right-click a port-21 packet → Follow > TCP Stream) shows a full authenticated FTP session: 220 (vsFTPd 3.0.3) → USER test_user → 331 Please specify the password. → PASS Nipun@123 → 230 Login successful. → CWD Test → 250 Directory successfully changed. → TYPE I / PASV / STOR <filename> → 226 Transfer complete. Conclusion: the infected host exfiltrates data via plaintext FTP to 140.82.59.185:21 — the confirmed C2/exfiltration server. No DNS query for this IP appears anywhere in the capture, meaning the address is hard-coded into the keylogger rather than resolved by hostname (reduces its forensic/DNS footprint).

```
ip.src == 192.168.76.131 && !(tcp.port == 80 || tcp.port == 443)
```

7. Frequency of Data Transmission

CONFIRMED: Two complete FTP upload cycles are visible in this ~85-second capture: the control-channel SYN to 140.82.59.185 occurs at $t=0.07s$ (frame 10) and again at $t=63.40s$ (frame 1795) — almost exactly 63 seconds apart. Each cycle uploads one report file and then closes the connection. This matches Ardamax Keylogger's configurable “send logs every N seconds/minutes” delivery option, set here to roughly a 1-minute interval. Filtering `ip.addr == 192.168.76.131 && ip.addr == 140.82.59.185` in Statistics > IO Graph shows two isolated ~1-second bursts of activity separated by ~62 seconds of silence — a clear beaconing pattern rather than continuous traffic.

8. Additional Data Exfiltrated (beyond keystrokes)

CONFIRMED: The two delivered files are not raw keystroke text — they are Ardamax-style HTML activity reports (full content and recovery method in Section 10). `Keys_2018-11-28_16-04-42.html` logs a timestamped active-window entry (‘28 November 2018 [16:04] explorer.exe: Pictures’) alongside the keystroke/window content, and contains the tool-identifying string “Ardamax_FTP_Delivery”. `Web_2018-11-29_11-28-13.html` logs browser activity: a timestamped active-window entry identifying the logged-in user (‘11:28 [29 November 2018] : nipun : Start - Microsoft Edge’) and the visited URL (`http://gmail.com/`). So beyond raw keystrokes, the malware also exfiltrates active window/application titles, timestamps, the logged-on Windows username (“nipun”), and visited website URLs — i.e. full user-activity monitoring, not just a keystroke stream.

9. Attacker Identification

CONFIRMED: The attacker's drop server is 140.82.59.185, running vsFTPD 3.0.3 on port 21. The malware authenticates with hard-coded credentials captured in cleartext from the FTP control stream — username `test_user`, password `Nipun@123` — uploading into a folder named `Test`. The string “Ardamax_FTP_Delivery” embedded in the first log file identifies the specific tool as Ardamax Keylogger configured in FTP-delivery mode. No SMTP/e-mail traffic appears in this capture (no port 25), so no attacker e-mail address is recoverable — the FTP credentials and drop-server IP are the direct attacker-infrastructure indicators available here. The infected host's Ethernet source MAC (`00:0c:29:ae:bf:e3`) carries a VMware OUI, confirming the victim host itself is a VM, consistent with this being a lab capture.

10. Extraction & Reconstruction of Exfiltrated Files

CONFIRMED: File > Export Objects > FTP-DATA lists two objects, both recovered intact: `Keys_2018-11-28_16-04-42.html` (579 bytes, keystroke/active-window log) and `Web_2018-11-29_11-28-13.html` (621 bytes, browsed-URL/active-window log). Both were reconstructed with the CLI equivalent of File > Export Objects — `tshark -r "Noobs Keylogger.pcap" --export-objects "ftp-data,extracted_files"` — and saved to `Network-Security/extracted_files/`. Opening them confirms both are well-formed Ardamax-style HTML reports (content quoted in Section 8).

11. Conclusion

Host 192.168.76.131 (a VMware VM) is infected with Ardamax Keylogger configured for FTP-based delivery. Approximately every 63 seconds the malware opens a plaintext FTP session to 140.82.59.185:21 (vsFTPD 3.0.3), authenticates as `test_user/Nipun@123`, and uploads an HTML activity report into a `/Test` directory on the server — alternating between a keystroke/active-window log (`Keys_*.html`) and a browsed-URL log (`Web_*.html`) — together revealing not just keystrokes but the logged-in username (“nipun”), application usage, and browsing activity. Both exfiltrated files were fully recovered from the capture (see `extracted_files/`). Because credentials and the drop path travel in cleartext FTP, and the C2 IP is hard-coded with no DNS lookup, the same artifacts (IP 140.82.59.185, port 21, credentials `test_user/Nipun@123`) can be used directly as network IOCs. Recommended remediation: remove the keylogger from the host, block 140.82.59.185 at the perimeter, force a credential reset for the affected user, and deploy egress filtering/DLP to flag outbound FTP or ~1-minute-interval beaconing to non-corporate IPs.

Assignment 2 — UFW Firewall Configuration: Basic Traffic Control & Logging

1. Objective

Configure UFW (Uncomplicated Firewall) on a Linux VM to detect and control two common attack behaviours — ICMP ping sweeps and TCP SYN scans (e.g. Nmap) — and to log suspicious activity rather than silently ignoring it.

2. Environment Setup

```
sudo apt update
sudo apt install ufw -y
sudo ufw enable
sudo ufw logging on
```

3. Ruleset (ufw_rules.txt)

Command	Purpose / Rationale
sudo ufw logging on	Enables logging of allowed and denied connections to /var/log/ufw.log so scan and brute-force activity is recorded, not just silently dropped.
sudo ufw limit 22/tcp	Rate-limits SSH: any host making more than 6 connection attempts within 30 seconds is temporarily denied. This slows or blocks brute-force login attempts while still allowing legitimate access, and every limited attempt is logged.
sudo ufw allow 80/tcp sudo ufw allow 443/tcp	Keeps essential web services reachable, establishing a baseline of 'normal' allowed traffic to contrast against scan noise on other ports.
sudo ufw deny 23/tcp	Telnet is insecure and a common scan target. Explicitly denying it (rather than leaving it simply unopened) generates a clear logged DENY entry whenever it is probed, useful evidence of reconnaissance.
sudo ufw default deny incoming sudo ufw default allow outgoing	Closes every port not explicitly allowed. An Nmap SYN scan sweeping many ports will now generate a burst of logged DENY entries from a single source IP in a short window, making the scan clearly visible in the logs.

Full command sequence, saved as ufw_rules.txt for submission:

```
sudo ufw enable
sudo ufw logging on
sudo ufw limit 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw deny 23/tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

4. Verifying the Configuration

```
sudo ufw status verbose
```

ACTUAL OUTPUT (captured from a live test rig — two Docker containers on an isolated bridge network, victim running this exact ruleset + OpenSSH, attacker running Nmap/SSH client): Status: active | Logging: on (low) | Default: deny (incoming), allow (outgoing), deny (routed) | 22/tcp LIMIT IN Anywhere | 80/tcp

ALLOW IN Anywhere | 443/tcp ALLOW IN Anywhere | 23/tcp DENY IN Anywhere (plus IPv6 equivalents). Full output saved to ufw_test_output/ufw_status_verbose.txt.

```

labuser@ufwlab-victim: ~ - sudo ufw status verbose

$ sudo ufw status verbose

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp LIMIT IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
23/tcp DENY IN Anywhere
22/tcp (v6) LIMIT IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
23/tcp (v6) DENY IN Anywhere (v6)

```

Figure 4 — `sudo ufw status verbose` on the live test rig, showing the ruleset from `ufw_rules.txt` active.

5. Testing

SSH brute-force simulation, from a second machine on the network:

```
for i in {1..8}; do ssh user@<UFW_VM_IP>; done
```

Nmap SYN scan simulation, from a second machine:

```
nmap -sS <UFW_VM_IP>
```

On the UFW VM, monitor the log live during both tests:

```
sudo tail -f /var/log/ufw.log
```

ACTUAL RESULTS from the live test rig: (a) Nmap SYN scan (`nmap -sS -p 21,22,23,80,443,3389,8080` against the victim) returned: 22/tcp open, 80/tcp & 443/tcp closed (allowed but nothing listening), 23/tcp filtered (explicit DENY, silently dropped), 21/3389/8080 filtered (caught by default-deny). (b) 8 rapid SSH connection attempts: the first 5 reached `sshd` and were rejected on password ("Permission denied"), then UFW's rate limiter tripped and attempts 6-8 got "Connection refused" before reaching `sshd`. Confirmed via `iptables` packet counters on the victim: the recent-match module hit `hit_count:6` within the 30s window, after which 3 packets were diverted to the LIMIT chain and REJECTed (`icmp-port-unreachable`) — exactly matching the 3 refused client connections. Full transcripts: `ufw_test_output/nmap_syn_scan.txt`, `ssh_bruteforce_client_output.txt`, `iptables_ssh_limit_counters.txt`.

```

attacker@ufwlab-attacker: ~ — nmap -sS scan

$ docker exec ufwlab-attacker nmap -sS -p 21,22,23,80,443,3389,8080 172.19.0.2

Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for ufwlab-victim.ufwlab-net (172.19.0.2)
Host is up (0.00016s latency).

PORT      STATE      SERVICE
21/tcp    filtered  ftp
22/tcp    open      ssh
23/tcp    filtered  telnet
80/tcp    closed    http
443/tcp   closed    https
3389/tcp  filtered  ms-wbt-server
8080/tcp  filtered  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 1.54 seconds

```

Figure 5 — (a) Nmap SYN scan (`nmap -sS`) against the victim container: port 22 open, 80/443 closed (allowed, nothing listening), 23 filtered (explicit deny), 21/3389/8080 filtered (default-deny).

```

attacker@ufwlab-attacker: ~ — SSH brute-force simulation (8 attempts)

$ docker exec ufwlab-attacker bash -c 'for i in $(seq 1 8); do
  sshpass -p "wrongpass" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=2 \
    -o PreferredAuthentications=password labuser@172.19.0.2 "echo hi"
done'

Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
ssh: connect to host 172.19.0.2 port 22: Connection refused
ssh: connect to host 172.19.0.2 port 22: Connection refused
ssh: connect to host 172.19.0.2 port 22: Connection refused

# First 5 attempts reached sshd (wrong password rejected).
# Attempts 6-8 were REJECTEd by UFW's rate limiter before reaching sshd.

```

Figure 6 — (b) SSH brute-force simulation: 8 rapid attempts from the attacker container; attempts 6-8 rejected once UFW's rate limiter tripped (matches the `iptables hit-count` evidence in `iptables_ssh_limit_counters.txt`).

6. Expected vs Observed Outcomes

Both behaved exactly as expected. The SSH rule allowed the first several connection attempts through (preserving normal access) and then automatically rate-limited further attempts from the same source within the 30-second window — verified by the recent-match hit-count reaching 6 and the subsequent packets being logged with the "[UFW LIMIT BLOCK]" prefix and REJECTEd. The default-deny policy correctly hid every port that wasn't explicitly allowed from the Nmap scan (filtered/closed rather than open), while the explicit deny on port 23 produced a clean, immediate drop distinguishable from the default-deny ports. No deviations from the expected outcome were observed.

7. Conclusion

This ruleset gives a minimal Linux host a working baseline defense against the two attack behaviours in scope: it slows/blocks SSH brute-forcing via UFW's built-in rate limiter while leaving normal access intact, and it denies-by-default so a port sweep (Nmap SYN scan) can't discover anything beyond the explicitly allowed services. Verified live against a real Nmap scan and a real repeated-SSH-connection test, both produced the expected LIMIT/DENY behaviour with no false negatives. Recommended follow-up hardening: fail2ban (or sshguard) layered on top of the UFW limit rule for IP-level banning after repeated failures,

restricting SSH to known source IPs/VPN ranges where feasible, and forwarding /var/log/ufw.log to a central SIEM for alerting rather than relying on local log review.